ASSURE DEFI®

THE VERIFICATION GOLD STANDARD

SECURITY ASSESSMENT REPORT

VERIFIED BY ASSURE DEFI®

INTEGRITY • TRUST • CREDIBILITY

NAME:	GG GUARDIAN
STATUS:	FAIL
DATE:	16/04/2026

Risk Analysis

Vulnerability summary

Classification	Description
High	Vulnerabilities that can lead to **loss or theft of funds**, **permanent locking of assets**, **unauthorized minting/burning**, **protocol insolvency**, or **full control of the contract or protocol**. These issues are typically **exploitable on-chain** and require immediate remediation.
Medium	Vulnerabilities that **weaken protocol security or trust assumptions** but do not directly enable immediate loss of funds under normal conditions. Exploitation may require **specific conditions, privileged roles, or external failures**.
Low	Issues that have **limited direct impact** on funds or protocol integrity but may cause **unexpected behavior**, **reduced resilience**, or **future exploitability** if combined with other weaknesses.
Informational	Findings that **do not represent security vulnerabilities**, but highlight **code quality, clarity, maintainability, or best-practice improvements** that may enhance long-term safety and auditability.

SCOPE

Target Code And Revision

Project	Assure
Codebase	https://github.com/GlobalGoldFinance/gg_guardian_contracts Commit: 4dde7d435733db3baa32b0045a7dabebb2c74ad7
Audit Methodology	Static, Manual

Detailed Technical Report

HIGH

1. Anyone can upgrade GuardianRouter and seize full control of governance

Location:

contracts/enterprise/GuardianRouter.sol:55-61

contracts/enterprise/GuardianRouter.sol:635-639

Issue:

GuardianRouter inherits UUPSUpgradeable, so proxy upgrades are gated by _authorizeUpgrade().

But the implementation is empty:

```
function _authorizeUpgrade(address newImplementation) internal override {
    // Authorization is handled by Guardian Enterprise base via governance
    // This function is called after governance approval
}
```

In UUPS, comments and external governance assumptions do not enforce authorization. The actual upgrade gate is _authorizeUpgrade(), and here it performs no check at all.

So if GuardianRouter is deployed behind a UUPS proxy, any caller can invoke the proxy upgrade entrypoint and swap in an attacker-controlled implementation.

Remediation:

Enforce authorization in _authorizeUpgrade().

Recommended pattern:

only allow upgrades when the router executes a governance-approved self-call,

require msg.sender == address(this) inside _authorizeUpgrade().

2. Uninitialized default guardian threshold enables zero-approval execution by default

Location:

contracts/utils/GuardianBase.sol:51-58

contracts/utils/GuardianBase.sol:412-428

contracts/utils/GuardianBase.sol:713-745

contracts/Guardian.sol:28-33

contracts/GuardianUpgradeable.sol:69-74

contracts/examples/Token.sol:39-41

contracts/examples/Token.sol:58-71

contracts/examples/Token.sol:119-160

Issue:

GuardianBase declares defaultGuardianThreshold but does not initialize it to a safe non-zero value during guardian-system setup.

The effective threshold resolution is:

```
function getGuardianThreshold(bytes32 actionId_) public view returns (uint8) {
    if (actionThresholdIsDefault[actionId_] || actionThresholds[actionId_] ==
0) {
        return defaultGuardianThreshold;
    }
    return actionThresholds[actionId_];
}
```

However, _initializeGuardianSystem() initializes guardian membership, queue state, and timeout, but does not assign defaultGuardianThreshold:

```
function _initializeGuardianSystem(address owner_, address[] memory
guardians_, uint256 maxGuardians_) internal {
    ...
    nextExecutableRequestId = 1;
    requestTimeout = 7 days;
}
```

Both wrapper initializers rely on this function:

Guardian.__Guardian_init(...)

GuardianUpgradeable.__GuardianUpgradeable_init(...)

Neither initializer requires a threshold parameter nor sets defaultGuardianThreshold afterward.

As a result, defaultGuardianThreshold remains 0 unless an integrating contract explicitly changes it later.

This becomes security-critical because _submitAction() auto-finalizes actions whose effective threshold is zero:

```
if (getGuardianThreshold(actionId_) == 0 && id == nextExecutableRequestId) {
    _finalizeAction(id);
}
```

Therefore, any guarded action that relies on the default threshold can execute immediately with zero guardian approvals after initialization.

This is not limited to the base contract in isolation: the issue propagates through the standard wrapper initialization flow and into example integrations that rely on default guardian-threshold behavior.

Remediation:

Make the default guardian threshold an explicit initialization parameter, or set a safe non-zero default during initialization.

Recommended fixes:

add defaultGuardianThreshold_ as an initializer parameter,

validate it against the configured guardian count,

only allow 0 when zero-approval execution is explicitly intended.

Example direction:

```
function _initializeGuardianSystem(
    address owner_,
    address[] memory guardians_,
    uint256 maxGuardians_,
    uint8 defaultGuardianThreshold_
) internal {
    ...
    require(defaultGuardianThreshold_ > 0, "invalid default threshold");
    require(defaultGuardianThreshold_ <= guardians_.length, "threshold too
high");
    defaultGuardianThreshold = defaultGuardianThreshold_;
    nextExecutableRequestId = 1;
    requestTimeout = 7 days;
}
```

Also add tests proving that:

a freshly initialized guardian contract cannot execute protected actions without approvals

unless zero-threshold behavior was explicitly chosen,

wrapper initializers do not leave the guardian system in an

approval-disabled state.

3. Enterprise governance deadlocks immediately after standard initialization

Location:

contracts/enterprise/GuardianEnterprise.sol:34-35

contracts/enterprise/GuardianEnterpriseUpgradeable.sol:171-174

contracts/enterprise/RoleRegistry.sol:164-192

contracts/enterprise/GuardianRouter.sol:149-157

Issue:

Both enterprise wrappers initialize the registry with exactly one admin:

```
function initializeInRegistry(address admin_) external onlyOwner {
    roleRegistry.initializeContract(admin_);
}
```

RoleRegistry.initializeContract() then:

adds exactly one ROLE_ADMIN member,

sets the default threshold to 1.

But GuardianRouter.approveRequest() forbids the submitter from approving their own request:

```
if (msg.sender == request.submitter) {
    revert CannotApproveOwnRequest();
}
```

The standard bootstrap path initializes the registry with a single admin and threshold 1, while GuardianRouter.approveRequest() forbids self-approval by the submitter. Under that configuration, router-governed requests submitted by the only admin cannot reach quorum unless there is some separate out-of-band path to add another approver. This creates a governance deadlock in the standard one-admin initialization flow.

Remediation:

Do not initialize the system with a single approver when self-approval is prohibited.

Safer options:

seed at least two admins initially,

allow self-approval when threshold is 1,

or split proposer and approver roles so there is always at least one independent approver.

Also add deployment-time validation that rejects impossible governance configurations.

4. GuardianRouter cannot govern itself because its execution pointer is never initialized

Location:

contracts/enterprise/GuardianRouter.sol:55-61

contracts/enterprise/GuardianRouter.sol:96-101

contracts/enterprise/GuardianRouter.sol:174-191

contracts/enterprise/GuardianRouter.sol:327-365

contracts/enterprise/GuardianRouter.sol:548-589

Issue:

During router initialization, the code sets only the router's timeout:

```
_governance[address(this)].requestTimeout = DEFAULT_TIMEOUT;
```

But it does not initialize:

```
_governance[address(this)].nextExecutableRequestId
```

Later, submitRequest() only initializes nextExecutableRequestId = 1 when requestTimeout == 0:

```
if (_governance[targetContract_].requestTimeout == 0) {
    _governance[targetContract_].requestTimeout = DEFAULT_TIMEOUT;
    _governance[targetContract_].nextExecutableRequestId = 1;
}
```

For address(this), that branch is skipped forever because requestTimeout was already set during initialization.

So for router-targeted requests:

the first request gets ID 1,

but nextExecutableRequestId remains 0.

Execution order logic then breaks:

approveRequest() only auto-executes when the request is the next executable one,

executeRequest() checks ordering against the same pointer,

_tryExecuteNext() starts scanning from request 0, finds no real request there, and stops.

Remediation:

Initialize router self-governance fully in initialize():

```
_governance[address(this)].requestTimeout = DEFAULT_TIMEOUT;
_governance[address(this)].nextExecutableRequestId = 1;
```

Also add a defensive backfill in submitRequest() for cases where:

requestCount > 0,

but nextExecutableRequestId == 0.

That prevents similar queue initialization bugs from persisting silently.

5. Overlapping approver roles are double-counted, which can lock actions permanently

Location:

contracts/enterprise/interfaces/IRoleRegistry.sol:200-206

contracts/enterprise/RoleRegistry.sol:469-477

contracts/enterprise/RoleRegistry.sol:483-504

Issue:

The interface documents that getApproverCount() should return the number of unique addresses that can approve an action:

```
// Get count of unique approvers for threshold validation
function getApproverCount(address forContract_, bytes32 actionId_) external
view returns (uint256);
```

But the implementation does not deduplicate addresses across roles. It just sums the size of each approver role set:

```
function getApproverCount(address forContract_, bytes32 actionId_) public view
returns (uint256) {
    bytes32[] memory approverRoles = getApproverRoles(forContract_,
actionId_);
    uint256 totalCount = 0;
    for (uint256 i = 0; i < approverRoles.length; i++) {
        totalCount += _roleMembers[forContract_][approverRoles[i]].length();
    }
    return totalCount;
}
```

So if the same approver belongs to multiple approver roles, that address is counted multiple times.

However, router approvals are tracked per address, not per role. A person who belongs to two roles can still only approve once on-chain.

This creates a real mismatch between:

the reported approver count used for governance math, and

the actual number of distinct approvals that can ever be cast.

The problem propagates into disapproval threshold calculations via:

```
uint256 approverCount = getApproverCount(forContract_, actionId_);
return approverCount >= 2 ? uint8(approverCount - 1) : 1;
```

Remediation:

Make getApproverCount() return the number of distinct addresses across all approver roles.

Recommended fixes:

deduplicate members across approver roles before counting,

validate thresholds against distinct approver count rather than raw role-member totals,

apply the same distinct-accounting logic anywhere disapproval or safety checks rely on approver count.

A robust implementation should treat approval power as per address, because that is how approvals are actually recorded and enforced by the router.

6. Custom action configuration can silently drop the approval threshold to 0, enabling approval-less execution

Location:

contracts/enterprise/RoleRegistry.sol:252-267

contracts/enterprise/RoleRegistry.sol:326-341

contracts/enterprise/RoleRegistry.sol:349-360

contracts/enterprise/RoleRegistry.sol:555-566

contracts/enterprise/GuardianRouter.sol:123-131

contracts/enterprise/GuardianRouter.sol:548-589

contracts/enterprise/GuardianRouter.sol:613-618

Issue:

Several action-configuration setters mark an action as having custom config without setting a custom threshold:

setApproverRoles(...)

setSubmitterRoles(...)

setOutOfOrderExecution(...)

Each of them sets:

```
config.hasCustomConfig = true;
```

But they do not set config.threshold.

Later, getThreshold() uses hasCustomConfig as the switch for whether to read the action-specific threshold or fall back to the default threshold:

```
function getThreshold(address forContract_, bytes32 actionId_) external view
returns (uint8) {
    ActionConfig storage config = _actionConfigs[forContract_][actionId_];
    return config.hasCustomConfig ? config.threshold :
_defaultConfigs[forContract_].threshold;
}
```

So once any of those setters is called for an action, getThreshold() stops using the default threshold and instead returns the uninitialized per-action config.threshold, which is 0.

The router then treats threshold 0 as immediately executable. _isExecutable() only checks:

```
request.approvals >= roleRegistry.getThreshold(targetContract_,
request.actionId)
```

If threshold is 0, that condition is true with zero approvals.

Then _tryExecuteNext() executes the request as soon as it is next in queue.

Remediation:

Do not overload a single hasCustomConfig flag to mean both:

“this action has custom roles/flags”

and “this action has a custom threshold”

Recommended fix:

add a dedicated hasCustomThreshold flag

make getThreshold() fall back to the default threshold unless a custom threshold was explicitly set

For example:

```
return config.hasCustomThreshold ? config.threshold : _defaultConfigs[forContract_].threshold;
```

Also add invariant tests proving that:

editing submitter roles,

editing approver roles,

or toggling out-of-order execution

cannot change quorum unless threshold is explicitly modified.

7. Guardian removal can silently drop quorum to zero, disabling future approvals

Location:

contracts/utils/GuardianBase.sol:586-601

contracts/utils/GuardianBase.sol:713-745

Issue:

When a guardian is removed, _removeGuardian() blindly decrements the default threshold whenever it is nonzero:

```
if (defaultGuardianThreshold > 0) {
    defaultGuardianThreshold = defaultGuardianThreshold - 1;
}
```

This does not preserve the intended security policy and does not clamp to a still-safe value. It simply lowers quorum by one every time a guardian is removed.

That becomes dangerous when the system is operating with a valid threshold of 1. Removing a guardian then changes the default threshold from 1 to 0.

Later, _submitAction() treats threshold 0 as approval-less execution:

```
if (getGuardianThreshold(actionId_) == 0 && id == nextExecutableRequestId) {
    _finalizeAction(id);
}
```

So a legitimate guardian-removal proposal can silently transform future guarded actions from “requires approval” into “executes immediately with zero approvals.”

Remediation:

Do not overload a single hasCustomConfig flag to mean both:

Remediation:

Do not decrement thresholds blindly on guardian removal.

Safer approaches:

recompute threshold from governance policy,

clamp to the maximum still-feasible safe value,

require explicit threshold reconfiguration instead of implicit auto-lowering,

and review the guardianCount - 1 feasibility assumptions so they match the actual submission/approval model.

At minimum, guardian removal must never silently set the effective threshold to 0 unless zero-approval execution is explicitly intended.

MEDIUM

1.resetActionToDefaults() does not clear out-of-order execution, so emergency bypass can persist after "reset"

Location:

contracts/enterprise/RoleRegistry.sol:222-232

contracts/enterprise/RoleRegistry.sol:349-360

contracts/enterprise/RoleRegistry.sol:600-601

Issue:

setOutOfOrderExecution() sets:

```
config.allowOutOfOrder = allowed_;
config.hasCustomConfig = true;
```

The code comment even states:

```
// Actually, resetActionToDefaults CLEARS the entire struct, so yes.
```

But resetActionToDefaults() does not clear the whole struct. It only clears:

submitterRoles

approverRoles

threshold

hasCustomConfig

It does not clear:

allowOutOfOrder

And canExecuteOutOfOrder() reads that flag directly:

```
return _actionConfigs[forContract_][actionId_].allowOutOfOrder;
```

So an action can remain out-of-order enabled even after governance thinks it has been reset to defaults.

The docs reinforce the expectation that reset removes custom configuration:

docs/guardian-enterprise/roles-and-permissions.md:239-252

Remediation:

Clear the full action config during reset, including allowOutOfOrder.

Example direction:

```
delete _actionConfigs[msg.sender][actionId_];
```

or explicitly set:

```
config.allowOutOfOrder = false;
```

2. Enterprise threshold setters lack invariant checks and can permanently brick governance

Location:

contracts/enterprise/RoleRegistry.sol:238-246

contracts/enterprise/RoleRegistry.sol:315-320

Issue:

The docs explicitly claim threshold validation exists:

docs/guardian-enterprise/thresholds.md:100-120 says zero threshold is disallowed after bootstrap and threshold must be less than approver count.

docs/guardian-enterprise/design.md:937 says: “Cannot set threshold >= approver count.”

But the implementation performs no validation at all in either threshold setter:

setActionThreshold(...) just writes config.threshold = threshold_

setDefaultThreshold(...) just writes _defaultConfigs[msg.sender].threshold = threshold_

There is no check against:

threshold == 0

threshold >= approver count

threshold >= effective unique approvers after self-approval blocking

Because submitters cannot approve their own requests, thresholds that look superficially valid can still be impossible in practice.

Remediation:

Add onchain validation to both setters.

At minimum:

disallow threshold == 0 outside an explicit bootstrap-only path,

require threshold < effective approver count when submitters are also allowed to approve,

more conservatively, require threshold <= maxApprovalsAvailableAfterSelfVoteBlock.

3. Registry migration is unreachable through the shipped enterprise wrapper flow

Location:

contracts/enterprise/GuardianEnterprise.sol:30-35

contracts/enterprise/GuardianEnterpriseUpgradeable.sol:167-174

contracts/enterprise/GuardianEnterpriseBase.sol:123-139

contracts/enterprise/GuardianEnterpriseActionsInternal.sol:398-446

contracts/enterprise/RoleRegistry.sol:164-192

Issue:

The enterprise wrappers expose only a standard bootstrap helper:

```
function initializeInRegistry(address admin_) external onlyOwner {
    roleRegistry.initializeContract(admin_);
}
```

This initializes governance in the current registry only.

But registry migration requires the replacement registry to already be configured for the governed contract before cutover. In migrateRegistry(...), the code checks the new registry's admin count and rejects migration if it is not already seeded:

```
if (newAdminCount < 2) {
    revert NewRegistryNotConfigured();
}
```

The problem is that RoleRegistry.initializeContract(address admin_) initializes state for msg.sender, not for an arbitrary contract chosen by the caller.

So if an EOA or owner tries to prepare a fresh replacement registry by calling:

```
newRegistry.initializeContract(admin);
```

that initializes the caller's own slot in the new registry, not the governed contract's slot.

And the shipped wrappers do not expose any path that allows the governed contract itself to pre-initialize an arbitrary replacement registry before calling migration.

Remediation:

Add an explicit supported path to initialize a nominated replacement registry for the governed contract before migration.

Safer options:

expose a governed/bootstrap function that makes the governed contract initialize itself in the new registry,

allow migration to atomically seed the new registry as part of the same approved action,

or redesign the new-registry precondition so it can be satisfied without requiring a separate self-call path that the wrappers do not expose.

Also ensure the documentation matches the actual callable flow provided by the shipped wrappers.

LOW

No low issues were found.

INFORMATIONAL

No informational issues were found.

Testing coverage

During the testing phase, custom use cases were written to cover all the logic of contracts. *Check "Annexes" to see the testing code.*

```
  Runtime security - property-based tests
    ✔ preserves ERC20 supply and aggregate balances across randomized transfers (1889ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuar
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argum
6.0)
    ✔ preserves role membership truth across randomized enterprise add/remove actions (235ms)


  2 passing (2s)
```

```
  Runtime security - fuzzing tests
    ✔ rejects malformed or unauthorized token governance requests across varied inputs (596ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"byte
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contra
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid typ
6.0)
    ✔ rejects malformed enterprise router submissions (87ms)


  2 passing (686ms)
```

```
  Runtime security - invariant tests
    ✔ enforces sequential execution for guardian requests once threshold is raised (607ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"by
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid t
6.0)
    ✔ maintains the minimum-admin invariant in the role registry (101ms)


  2 passing (710ms)
```

```
  Runtime security - exploit reproduction tests
    ✓ blocks implementation initialization and proxy reinitialization on upgradeable guardian contracts (560ms)
    ✓ reproduces and blocks self-approval and double-vote attempts (40ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalT
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuardianRoute
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type
6.0)
    ✓ blocks router self-approval in enterprise governance (83ms)


  3 passing (688ms)
```

```
  Runtime security - reentrancy and authorization abuse simulations
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGu
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (arg
6.0)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGu
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (arg
6.0)
    ✓ marks enterprise requests executed before callback logic can reenter approval flow (661ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGu
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (arg
6.0)
    ✓ rejects direct unauthorized enterprise execution and approval abuse (79ms)


  2 passing (742ms)
```

```
  Runtime security - Guardian logic cybersecurity tests
    ✓ releases a queued approved request only after an expired head request is cancelled (553ms)
    ✓ auto-reduces threshold after guardian removal so governance remains live (44ms)
    ✓ revokes approval power immediately after guardian removal despite index swaps (48ms)
    ✓ uses disapproval cancellation to unblock a pre-approved queued request safely
    ✓ batch-cancels expired blockers and frees a later fully approved request


  5 passing (723ms)
```

Annexes

Testing code:

Property based runtime:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployNonEnterpriseFixture, deployEnterpriseFixture, randBetween } =
require('./runtime/helpers');

describe('Runtime security - property-based tests', function () {
  it('preserves ERC20 supply and aggregate balances across randomized transfers', async
function () {
    const { token, alice, bob, charlie } = await deployNonEnterpriseFixture();
    const users = [alice, bob, charlie];
    const initial = [1000n, 2000n, 3000n];

    for (let i = 0; i < users.length; i++) {
      await (await token.testMint(users[i].address, initial[i])).wait();
    }

    const initialSupply = await token.totalSupply();

    for (let i = 0; i < 40; i++) {
      const fromIdx = randBetween(`from-${i}`, 0, 2);
      let toIdx = randBetween(`to-${i}`, 0, 2);
      if (fromIdx === toIdx) toIdx = (toIdx + 1) % 3;

      const from = users[fromIdx];
      const to = users[toIdx];
      const bal = await token.balanceOf(from.address);
      const capped = bal > 100n ? 100 : Number(bal);
      const amount = bal === 0n ? 0n : BigInt(randBetween(`amt-${i}`, 0, capped));
      await (await token.connect(from).transfer(to.address, amount)).wait();
```

```
      const aggregate =
        await token.balanceOf(alice.address) +
        await token.balanceOf(bob.address) +
        await token.balanceOf(charlie.address);

      expect(await token.totalSupply()).to.equal(initialSupply);
      expect(aggregate).to.equal(initialSupply);
    }
  });

  it('preserves role membership truth across randomized enterprise add/remove actions',
async function () {
    const fx = await deployEnterpriseFixture();
    const { router, target, approver1, approver2, submitter, receiver, ACTION_CREATE_ROLE,
ACTION_ADD_MEMBER, ACTION_REMOVE_MEMBER } = fx;
    const ROLE_TESTER = ethers.keccak256(ethers.toUtf8Bytes('ROLE.TESTER'));
    const coder = ethers.AbiCoder.defaultAbiCoder();
    const states = [];

    async function submitAndExecute(actionId, data) {
      const tx = await router.connect(submitter).submitRequest(await target.getAddress(),
actionId, data);
      const receipt = await tx.wait();
      const ev = receipt.logs.map((l) => {
        try { return router.interface.parseLog(l); } catch { return null; }
      }).find((x) => x && x.name === 'RequestSubmitted');
      const requestId = ev.args.requestId;
      await (await router.connect(approver1).approveRequest(await target.getAddress(),
requestId)).wait();
      await (await router.connect(approver2).approveRequest(await target.getAddress(),
requestId)).wait();
      return requestId;
    }

    await submitAndExecute(ACTION_CREATE_ROLE, coder.encode(['bytes32'], [ROLE_TESTER]));

    for (let i = 0; i < 10; i++) {
      const shouldAdd = i % 2 === 0;
      if (shouldAdd) {
        await submitAndExecute(ACTION_ADD_MEMBER, coder.encode(['bytes32', 'address'],
[ROLE_TESTER, receiver.address]));
        states.push(true);
      } else {
        await submitAndExecute(ACTION_REMOVE_MEMBER, coder.encode(['bytes32', 'address'],
[ROLE_TESTER, receiver.address]));
```

```
        states.push(false);
      }
      expect(await fx.registry.hasRole(await target.getAddress(), ROLE_TESTER,
receiver.address)).to.equal(states[states.length - 1]);
    }
  });
});
```

Fuzzing runtime:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployNonEnterpriseFixture, deployEnterpriseFixture, expectRevert,
getEnterpriseFactories } = require('./runtime/helpers');

describe('Runtime security - fuzzing tests', function () {
  it('rejects malformed or unauthorized token governance requests across varied inputs',
async function () {
    const { token, owner, attacker, alice } = await deployNonEnterpriseFixture();
    const zero = ethers.ZeroAddress;
    const badAction = ethers.keccak256(ethers.toUtf8Bytes('BAD.ACTION'));
    const mintAction = await token.MINT_TOKENS();

    await expectRevert(token.connect(attacker).submitActionRequest(mintAction, [10],
[alice.address], []));

    await expectRevert(token.connect(owner).submitActionRequest(badAction, [], [], []));

    const cases = [
      { amount: 0, to: alice.address, ok: false },
      { amount: 1, to: zero, ok: false },
      { amount: 1, to: alice.address, ok: true },
      { amount: 7, to: alice.address, ok: true },
      { amount: 0, to: zero, ok: false },
    ];

    for (const c of cases) {
      const tx = token.connect(owner).submitActionRequest(mintAction, [c.amount], [c.to],
[]);
      if (c.ok) {
        await (await tx).wait();
      } else {
        await expectRevert(tx);
      }
    }
```

```
  });

  it('rejects malformed enterprise router submissions', async function () {
    const { router, target, submitter, outsider, ACTION_ADD_MEMBER } = await
deployEnterpriseFixture();
    const coder = ethers.AbiCoder.defaultAbiCoder();

    await expectRevert(router.connect(outsider).submitRequest(await target.getAddress(),
ACTION_ADD_MEMBER, coder.encode(['bytes32', 'address'], [ethers.ZeroHash,
outsider.address])));

    await expectRevert(router.connect(submitter).submitRequest(await target.getAddress(),
ethers.ZeroHash, '0x'));

    await expectRevert(router.connect(submitter).submitRequest(await target.getAddress(),
ACTION_ADD_MEMBER, '0x1234'));

    await expectRevert(router.connect(submitter).submitRequest(await target.getAddress(),
ACTION_ADD_MEMBER, coder.encode(['bytes32', 'address'],
[ethers.keccak256(ethers.toUtf8Bytes('ROLE.X')), ethers.ZeroAddress])));
  });
});
```

Invariant runtime:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployNonEnterpriseFixture, deployEnterpriseFixture, expectRevert } =
require('./runtime/helpers');

describe('Runtime security - invariant tests', function () {
  it('enforces sequential execution for guardian requests once threshold is raised', async
function () {
    const { token, owner, g1, g2, g3, attacker } = await deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    expect(await token.defaultGuardianThreshold()).to.equal(2n);

    await (await token.connect(owner).submitActionRequest(addGuardian, [],
[attacker.address], [])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [],
[ethers.Wallet.createRandom().address], [])).wait();
```

```
    await (await token.connect(g1).submitApproval(2)).wait();
    await (await token.connect(g2).submitApproval(2)).wait();

    expect(await token.isGuardian(attacker.address)).to.equal(true);
    expect(await token.nextExecutableRequestId()).to.equal(3n);

    await (await token.connect(g1).submitApproval(3)).wait();
    expect(await token.canExecuteRequest(3)).to.equal(false);
    await (await token.connect(g3).submitApproval(3)).wait();
    expect(await token.nextExecutableRequestId()).to.equal(4n);
  });

  it('maintains the minimum-admin invariant in the role registry', async function () {
    const fx = await deployEnterpriseFixture();
    const { router, target, approver1, approver2, submitter, admin, ROLE_ADMIN } = fx;
    const coder = ethers.AbiCoder.defaultAbiCoder();

    const ACTION_BATCH_REMOVE_MEMBERS =
ethers.keccak256(ethers.toUtf8Bytes('ACTION.BATCH_REMOVE_MEMBERS'));
    const tx = await router.connect(submitter).submitRequest(
      await target.getAddress(),
      ACTION_BATCH_REMOVE_MEMBERS,
      coder.encode(['bytes32', 'address[]'], [ROLE_ADMIN, [admin.address,
approver1.address, approver2.address]])
    );
    const receipt = await tx.wait();
    const parsed = receipt.logs.map((l) => { try { return router.interface.parseLog(l); }
catch { return null; } }).find((x) => x && x.name === 'RequestSubmitted');
    const requestId = parsed.args.requestId;

    await (await router.connect(approver1).approveRequest(await target.getAddress(),
requestId)).wait();
    await expectRevert(router.connect(approver2).approveRequest(await target.getAddress(),
requestId));

    expect(await fx.registry.hasRole(await target.getAddress(), ROLE_ADMIN,
admin.address)).to.equal(true);
  });
});
```

Exploit reproduction:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployNonEnterpriseFixture, deployEnterpriseFixture, expectRevert,
getEnterpriseFactories } = require('./runtime/helpers');

describe('Runtime security - exploit reproduction tests', function () {
  it('blocks implementation initialization and proxy reinitialization on upgradeable
guardian contracts', async function () {
    const [owner, g1, g2] = await ethers.getSigners();
    const Impl = await ethers.getContractFactory('TestGuardianUpgradeable');
    const impl = await Impl.deploy();
    await impl.waitForDeployment();

    await expectRevert(impl.initialize(owner.address, [g1.address, g2.address]));

    const Proxy = await ethers.getContractFactory('GenericProxy');
    const initData = Impl.interface.encodeFunctionData('initialize', [owner.address,
[g1.address, g2.address]]);
    const proxy = await Proxy.deploy(await impl.getAddress(), initData);
    await proxy.waitForDeployment();
    const proxied = Impl.attach(await proxy.getAddress());

    expect(await proxied.owner()).to.equal(owner.address);
    await expectRevert(proxied.initialize(owner.address, [g1.address, g2.address]));
  });

  it('reproduces and blocks self-approval and double-vote attempts', async function () {
    const { token, owner, g1, g2, alice } = await deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const mintAction = await token.MINT_TOKENS();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(mintAction, [5], [alice.address],
[])).wait();

    await expectRevert(token.connect(owner).submitApproval(2));
```

```
    await (await token.connect(g1).submitApproval(2)).wait();
    await expectRevert(token.connect(g1).submitApproval(2));
    await (await token.connect(g2).submitApproval(2)).wait();
    expect(await token.balanceOf(alice.address)).to.equal(5n);
  });

  it('blocks router self-approval in enterprise governance', async function () {
    const fx = await deployEnterpriseFixture();
    const { router, target, submitter, approver1, approver2, ACTION_ADD_MEMBER, receiver,
ROLE_ADMIN } = fx;
    const coder = ethers.AbiCoder.defaultAbiCoder();
    const tx = await router.connect(submitter).submitRequest(
      await target.getAddress(),
      ACTION_ADD_MEMBER,
      coder.encode(['bytes32', 'address'], [ROLE_ADMIN, receiver.address])
    );
    const receipt = await tx.wait();
    const parsed = receipt.logs.map((l) => { try { return router.interface.parseLog(l); }
catch { return null; } }).find((x) => x && x.name === 'RequestSubmitted');
    const requestId = parsed.args.requestId;

    await expectRevert(router.connect(submitter).approveRequest(await target.getAddress(),
requestId));
    await (await router.connect(approver1).approveRequest(await target.getAddress(),
requestId)).wait();
    await (await router.connect(approver2).approveRequest(await target.getAddress(),
requestId)).wait();
    expect(await fx.registry.hasRole(await target.getAddress(), ROLE_ADMIN,
receiver.address)).to.equal(true);
  });
});
```

Reentrancy runtime:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployEnterpriseFixture, expectRevert, getEnterpriseFactories } =
require('./runtime/helpers');

describe('Runtime security - reentrancy and authorization abuse simulations', function () {
  it('marks enterprise requests executed before callback logic can reenter approval flow',
async function () {
    const fx = await deployEnterpriseFixture();
    const { registry, router, admin, approver1, approver2, submitter, receiver,
ACTION_ADD_MEMBER, ROLE_ADMIN } = fx;
    const coder = ethers.AbiCoder.defaultAbiCoder();

    const { Reentrant } = await getEnterpriseFactories();
    const target = await Reentrant.deploy();
    await target.waitForDeployment();
    await (await target.initialize(await registry.getAddress(), await
router.getAddress())).wait();
    await (await target.initializeInRegistry(admin.address)).wait();

    const irole = new ethers.Interface([
      'function addMember(bytes32 roleId_, address member_)',
      'function setDefaultThreshold(uint8 threshold_)'
    ]);
    await (await target.callRegistry(irole.encodeFunctionData('addMember', [ROLE_ADMIN,
approver1.address]))).wait();
    await (await target.callRegistry(irole.encodeFunctionData('addMember', [ROLE_ADMIN,
approver2.address]))).wait();
    await (await target.callRegistry(irole.encodeFunctionData('addMember', [ROLE_ADMIN,
submitter.address]))).wait();
    await (await target.callRegistry(irole.encodeFunctionData('addMember', [ROLE_ADMIN,
await target.getAddress()]))).wait();
    await (await target.callRegistry(irole.encodeFunctionData('setDefaultThreshold',
[2]))).wait();

    const tx = await router.connect(submitter).submitRequest(
      await target.getAddress(),
```

```
      ACTION_ADD_MEMBER,
      coder.encode(['bytes32', 'address'], [ROLE_ADMIN, receiver.address])
    );
    const receipt = await tx.wait();
    const parsed = receipt.logs.map((l) => { try { return router.interface.parseLog(l); }
catch { return null; } }).find((x) => x && x.name === 'RequestSubmitted');
    const requestId = parsed.args.requestId;

    await (await target.configureReentrancy(requestId, true)).wait();
    await (await router.connect(approver1).approveRequest(await target.getAddress(),
requestId)).wait();
    await (await router.connect(approver2).approveRequest(await target.getAddress(),
requestId)).wait();

    const pending = await router.getPendingRequest(await target.getAddress(), requestId);
    expect(pending[3]).to.equal(true);
    expect(await target.attempted()).to.equal(true);
    expect(await registry.hasRole(await target.getAddress(), ROLE_ADMIN,
receiver.address)).to.equal(true);
  });

  it('rejects direct unauthorized enterprise execution and approval abuse', async function
() {
    const fx = await deployEnterpriseFixture();
    const { router, target, submitter, outsider, ACTION_ADD_MEMBER, receiver, ROLE_ADMIN }
= fx;
    const coder = ethers.AbiCoder.defaultAbiCoder();

    await expectRevert(target.connect(outsider).executeAction(ACTION_ADD_MEMBER,
coder.encode(['bytes32', 'address'], [ROLE_ADMIN, receiver.address])));

    const tx = await router.connect(submitter).submitRequest(
      await target.getAddress(),
      ACTION_ADD_MEMBER,
      coder.encode(['bytes32', 'address'], [ROLE_ADMIN, receiver.address])
    );
    const receipt = await tx.wait();
    const parsed = receipt.logs.map((l) => { try { return router.interface.parseLog(l); }
catch { return null; } }).find((x) => x && x.name === 'RequestSubmitted');
    const requestId = parsed.args.requestId;

    await expectRevert(router.connect(outsider).approveRequest(await target.getAddress(),
requestId));
  });
});
```

Guardian logic runtime:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { deployNonEnterpriseFixture, expectRevert } = require('./runtime/helpers');

async function increaseTime(seconds) {
  await ethers.provider.send('evm_increaseTime', [seconds]);
  await ethers.provider.send('evm_mine', []);
}

describe('Runtime security - Guardian logic cybersecurity tests', function () {
  it('releases a queued approved request only after an expired head request is cancelled',
async function () {
    const { token, owner, g1, g2, alice, bob } = await deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [alice.address],
[])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [bob.address],
[])).wait();

    await (await token.connect(g1).submitApproval(3)).wait();
    await (await token.connect(g2).submitApproval(3)).wait();

    expect(await token.isGuardian(bob.address)).to.equal(false);
    expect(await token.nextExecutableRequestId()).to.equal(2n);

    await increaseTime(8 * 24 * 60 * 60);
    await (await token.cancelExpiredRequest(2)).wait();

    expect(await token.isGuardian(alice.address)).to.equal(false);
    expect(await token.isGuardian(bob.address)).to.equal(true);
    expect(await token.nextExecutableRequestId()).to.equal(4n);
  });

  it('auto-reduces threshold after guardian removal so governance remains live', async
function () {
```

```
    const { token, owner, g1, g2, g3, alice } = await deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const removeGuardian = await token.GUARDIAN_REMOVE();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(removeGuardian, [], [g3.address],
[])).wait();
    await (await token.connect(g1).submitApproval(2)).wait();
    await (await token.connect(g2).submitApproval(2)).wait();

    expect(await token.isGuardian(g3.address)).to.equal(false);
    expect(await token.defaultGuardianThreshold()).to.equal(1n);

    await (await token.connect(owner).submitActionRequest(addGuardian, [], [alice.address],
[])).wait();
    await (await token.connect(g1).submitApproval(3)).wait();

    expect(await token.isGuardian(alice.address)).to.equal(true);
    expect(await token.nextExecutableRequestId()).to.equal(4n);
  });

  it('revokes approval power immediately after guardian removal despite index swaps', async
function () {
    const { token, owner, g1, g2, g3, alice } = await deployNonEnterpriseFixture();
    const removedGuardian = g2.address;
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const removeGuardian = await token.GUARDIAN_REMOVE();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(removeGuardian, [],
[removedGuardian], [])).wait();
    await (await token.connect(g1).submitApproval(2)).wait();
    await (await token.connect(g3).submitApproval(2)).wait();

    expect(await token.isGuardian(removedGuardian)).to.equal(false);
    expect(await token.guardianIndex(removedGuardian)).to.equal(0n);

    await (await token.connect(owner).submitActionRequest(addGuardian, [], [alice.address],
[])).wait();
    await expectRevert(token.connect(g2).submitApproval(3));

    await (await token.connect(g1).submitApproval(3)).wait();
    expect(await token.isGuardian(alice.address)).to.equal(true);
```

```
  });

  it('uses disapproval cancellation to unblock a pre-approved queued request safely', async
function () {
    const { token, owner, g1, g2, alice, bob } = await deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [alice.address],
[])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [bob.address],
[])).wait();

    await (await token.connect(g1).submitApproval(3)).wait();
    await (await token.connect(g2).submitApproval(3)).wait();
    expect(await token.isGuardian(bob.address)).to.equal(false);

    await (await token.connect(g1).submitDisapproval(2)).wait();
    await (await token.connect(g2).submitDisapproval(2)).wait();

    const blocked = await token.getRequestState(2);
    expect(blocked.finalized).to.equal(true);
    expect(await token.isGuardian(alice.address)).to.equal(false);
    expect(await token.isGuardian(bob.address)).to.equal(true);
    expect(await token.nextExecutableRequestId()).to.equal(4n);
  });

  it('batch-cancels expired blockers and frees a later fully approved request', async
function () {
    const { token, owner, g1, g2, charlie, alice, bob } = await
deployNonEnterpriseFixture();
    const setDefault = await token.GUARDIAN_SET_DEFAULT_THRESHOLD();
    const addGuardian = await token.GUARDIAN_ADD();

    await (await token.connect(owner).submitActionRequest(setDefault, [2], [], [])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [alice.address],
[])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [], [bob.address],
[])).wait();
    await (await token.connect(owner).submitActionRequest(addGuardian, [],
[charlie.address], [])).wait();

    await (await token.connect(g1).submitApproval(4)).wait();
    await (await token.connect(g2).submitApproval(4)).wait();
```

```
    expect(await token.isGuardian(charlie.address)).to.equal(false);

    await increaseTime(8 * 24 * 60 * 60);
    await (await token.batchCancelExpiredRequests([2, 3])).wait();

    expect(await token.isGuardian(alice.address)).to.equal(false);
    expect(await token.isGuardian(bob.address)).to.equal(false);
    expect(await token.isGuardian(charlie.address)).to.equal(true);
    expect(await token.nextExecutableRequestId()).to.equal(5n);
  });
});
```

Foundry POC:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {GuardianEnterprise} from "../enterprise/GuardianEnterprise.sol";
import {GuardianRouter} from "../enterprise/GuardianRouter.sol";
import {RoleRegistry} from "../enterprise/RoleRegistry.sol";
import {
    CannotApproveOwnRequest,
    InvalidRequestOrder,
    NotAuthorizedApprover
} from "../enterprise/utils/GuardianEnterpriseErrors.sol";
import {Token} from "../examples/Token.sol";

interface Vm {
    function expectRevert(bytes4 revertData) external;
    function expectRevert(bytes calldata revertData) external;
    function prank(address msgSender) external;
}

contract TestRoleRegistry is RoleRegistry {}

contract DummyCode {}

contract HarnessRouter is GuardianRouter {
    function bootstrapInitializeInRegistry(address admin_) external {
        roleRegistry.initializeContract(admin_);
    }

    function bootstrapAddAdmin(address member_) external {
        roleRegistry.addMember(roleRegistry.roleAdmin(), member_);
    }
```

```
}

contract HarnessEnterprise is GuardianEnterprise {
    constructor(address roleRegistry_, address guardianRouter_)
GuardianEnterprise(roleRegistry_, guardianRouter_) {}

    function bootstrapInitializeInRegistry(address admin_) external {
        roleRegistry.initializeContract(admin_);
    }

    function bootstrapCreateRole(bytes32 roleId_) external {
        roleRegistry.createRole(roleId_);
    }

    function bootstrapAddMember(bytes32 roleId_, address member_) external {
        roleRegistry.addMember(roleId_, member_);
    }
}

contract GuardianPoCTest {
    Vm internal constant vm = Vm(address(uint160(uint256(keccak256("hevm cheat
code")))));

    address internal constant ADMIN1 = address(0xA11CE);
    address internal constant ADMIN2 = address(0xB0B);
    address internal constant ATTACKER = address(0xBAD);
    address internal constant OWNER = address(0xABCD);
    address internal constant GUARDIAN1 = address(0x1111);
    address internal constant GUARDIAN2 = address(0x2222);
    address internal constant RECIPIENT = address(0xCAFE);

    bytes32 internal constant OPS_ROLE = keccak256("ROLE.OPS");

    function testSingleAdminBootstrapDeadlocksGovernance() public {
        (TestRoleRegistry registry, HarnessRouter router) = _deployRouter();
        GuardianEnterprise target = new GuardianEnterprise(address(registry),
address(router));
        bytes32 actionId = target.ACTION_CREATE_ROLE();

        target.initializeInRegistry(ADMIN1);

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(target), actionId,
abi.encode(OPS_ROLE));

        vm.prank(ADMIN1);
        vm.expectRevert(CannotApproveOwnRequest.selector);
        router.approveRequest(address(target), requestId);
```

```
        vm.prank(ATTACKER);
        vm.expectRevert(NotAuthorizedApprover.selector);
        router.approveRequest(address(target), requestId);

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(target), requestId);
        _assertFalse(executed, "request should remain pending");
        _assertEq(approvalCount, 0, "single admin should not be able to
self-approve");
    }

    function testRouterSelfGovernanceRequestsCannotExecute() public {
        (, HarnessRouter router) = _deployRouter();
        bytes32 actionId = router.ACTION_SET_REQUEST_TIMEOUT();

        router.bootstrapInitializeInRegistry(ADMIN1);
        router.bootstrapAddAdmin(ADMIN2);

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(router), actionId,
abi.encode(uint256(2 hours)));

        vm.prank(ADMIN2);
        router.approveRequest(address(router), requestId);

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(router), requestId);
        _assertFalse(executed, "router self-request should still be pending");
        _assertEq(approvalCount, 1, "request should have one approval");

        vm.expectRevert(abi.encodeWithSelector(InvalidRequestOrder.selector,
requestId));
        router.executeRequest(address(router), requestId);

        _assertEq(
            router.getRequestTimeout(address(router)),
            router.DEFAULT_TIMEOUT(),
            "timeout should stay unchanged when self-governance is stuck"
        );
    }

    function testCustomSubmitterRolesDropThresholdToZero() public {
        (TestRoleRegistry registry, HarnessRouter router) = _deployRouter();
        HarnessEnterprise target = new HarnessEnterprise(address(registry),
address(router));
        DummyCode maliciousRouter = new DummyCode();
        bytes32 configActionId = target.ACTION_SET_SUBMITTER_ROLES();
        bytes32 targetActionId = target.ACTION_UPDATE_ROUTER();
```

```
        target.bootstrapInitializeInRegistry(ADMIN1);
        target.bootstrapAddMember(registry.roleAdmin(), ADMIN2);
        target.bootstrapCreateRole(OPS_ROLE);
        target.bootstrapAddMember(OPS_ROLE, ATTACKER);

        vm.prank(ADMIN1);
        uint256 configRequestId = router.submitRequest(
            address(target),
            configActionId,
            abi.encode(targetActionId, _bytes32Array1(OPS_ROLE))
        );

        vm.prank(ADMIN2);
        router.approveRequest(address(target), configRequestId);

        _assertEq(
            registry.getThreshold(address(target), targetActionId),
            0,
            "custom role config should have left a zero threshold"
        );

        vm.prank(ATTACKER);
        uint256 exploitRequestId = router.submitRequest(address(target),
targetActionId, abi.encode(address(maliciousRouter)));

        _assertEq(
            address(target.guardianRouter()),
            address(maliciousRouter),
            "attacker-submitted request should execute without approvals"
        );

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(target), exploitRequestId);
        _assertTrue(executed, "exploit request should auto-execute");
        _assertEq(approvalCount, 0, "exploit should require zero approvals");
    }

    function testGuardianInitializesWithZeroThresholdAndAutoExecutes() public {
        Token token = new Token("Guardian Token", "GTK", OWNER,
_addressArray2(GUARDIAN1, GUARDIAN2));
        bytes32 mintActionId = token.MINT_TOKENS();

        vm.prank(OWNER);
        token.submitActionRequest(
            mintActionId,
            _uint256Array1(100),
            _addressArray1(RECIPIENT),
            new bytes32[](0)
        );
```

```
        _assertEq(token.balanceOf(RECIPIENT), 100, "mint should have executed
immediately");

        (, , , uint8 approvalCount, , bool finalized, , , ) = token.getRequest(1);
        _assertEq(uint256(approvalCount), 0, "request should have zero guardian
approvals");
        _assertTrue(finalized, "request should already be finalized");
    }

    function _deployRouter() internal returns (TestRoleRegistry registry_,
HarnessRouter router_) {
        registry_ = new TestRoleRegistry();

        HarnessRouter implementation = new HarnessRouter();
        ERC1967Proxy proxy = new ERC1967Proxy(
            address(implementation),
            abi.encodeWithSelector(bytes4(keccak256("initialize(address)")),
address(registry_))
        );

        router_ = HarnessRouter(address(proxy));
    }

    function _addressArray1(address a_) internal pure returns (address[] memory arr_)
{
        arr_ = new address[](1);
        arr_[0] = a_;
    }

    function _addressArray2(address a_, address b_) internal pure returns (address[]
memory arr_) {
        arr_ = new address[](2);
        arr_[0] = a_;
        arr_[1] = b_;
    }

    function _bytes32Array1(bytes32 a_) internal pure returns (bytes32[] memory arr_)
{
        arr_ = new bytes32[](1);
        arr_[0] = a_;
    }

    function _uint256Array1(uint256 a_) internal pure returns (uint256[] memory arr_)
{
        arr_ = new uint256[](1);
        arr_[0] = a_;
    }
```

```
    function _assertEq(address actual_, address expected_, string memory message_)
internal pure {
        require(actual_ == expected_, message_);
    }

    function _assertEq(uint256 actual_, uint256 expected_, string memory message_)
internal pure {
        require(actual_ == expected_, message_);
    }

    function _assertFalse(bool condition_, string memory message_) internal pure {
        require(!condition_, message_);
    }

    function _assertTrue(bool condition_, string memory message_) internal pure {
        require(condition_, message_);
    }
}
```

Technical Findings Summary

Findings

Vulnerability Level	Total	Pending	Not Apply	Acknowledged	Partially Fixed	Fixed
High	7					
Medium	3					
Low						
Informational						

Assessment Results

Score Results

Review	Score
Global Score	**50/100**
Assure KYC	Not completed
Audit Score	50/100

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

Audit FAIL

Following our comprehensive security audit of the token contract for the **GG GUARDIAN** project, the project did not fulfill the necessary criteria required to pass the security audit.

Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided ‘as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.